ESTRUTURA DE DADOS

Subarray Contíguo de Soma Máxima

Encontrando o trecho mais valioso de um vetor com o algoritmo de Kadane

Mariana Montilha de Almeida e Mateus Ciffone



O que é o problema?

Dado um array de inteiros,, encontre o subarray com a maior soma, e retorne sua soma .

ENTRADA	Um array de inteiros (positivos, negativos e zero)
SAÍDA	A soma máxima encontrada (algumas variantes pedem também os índices de início e fim)
RESTRIÇÃO-CHAVE	O subvetor deve ser contíguo e ter pelo menos um elemento

EXEMPLO

Entrada:

```
[-2, 1, -3, 4, -1, 2, 1, -5, 4]
```

Saída (soma máxima):

6

Comparando as somas parciais à medida que percorremos o vetor, conseguimos identificar qual trecho contíguo maximiza o resultado.

Contexto em entrevistas técnicas

NÍVEL DE DIFICULDADE

Easy | **Medium** | Hard

- LeetCode #53 — classificado como Medium.
- Aparece com frequência em triagens de arrays por parecer simples - candidatos tentam força bruta e não acham a otimização.
- A dificuldade não está na sintaxe, e sim em enxergar o padrão de decisão local (programação dinâmica) antes de programar.

EMPRESAS QUE JÁ COBRARAM ESSA QUESTÃO

- **Amazon**
- **Google**
- **Meta**
- **Microsoft**
- **LinkedIn**

Com base em relatos de candidatos no LeetCode Discuss, Glassdoor e fóruns como Blind.

PARA QUAIS VAGAS? Entrevistas técnicas de Software/Backend Engineer, tipicamente em vagas de nível júnior a pleno — avalia raciocínio incremental sobre arrays, não a decoreba de um algoritmo.

Abordagens Incorretas

Força bruta (testar tudo)

• Testa toda combinação de índice inicial e final, somando os elementos do zero a cada tentativa.

• Funciona, mas recalcula a mesma soma repetidas vezes — nada do trabalho anterior é aproveitado.

$O(n^3)$

tempo — ainda revisita trechos já somados

Soma de prefixos

• Pré-calcula somas acumuladas: a soma de qualquer trecho vira uma subtração entre dois prefixos.

• Evita recalcular somas, mas ainda testa todo par (início, fim) possível para achar o melhor.

• Melhora o cálculo da soma, mas não resolve a explosão combinatória de pares.

$O(n^2)$

tempo — continua comparando todos os pares

Como o Kadane resolve o problema

A ideia: percorrer o vetor uma única vez, mantendo a soma do subarray que termina na posição atual e comparando-a, a cada passo, com o maior valor já encontrado.

1

Inicializar

max_atual e max_global recebem o primeiro elemento do vetor.

2

Continuar ou recomeçar

Para cada próximo elemento: se a soma acumulada ainda ajuda, estende-se o subarray; senão, recomeça-se a partir dali.

3

Atualizar o recorde

Compara-se max_atual com max_global e guarda-se o maior dos dois.

4

Retornar o resultado

Ao final do vetor, max_global contém a soma máxima possível.

Vendo o algoritmo em um caso concreto

Entrada: `nums = [-2, 1, -3, 4, -1, 2, 1, -5, 4]`



Subarray ideal: `[4, -1, 2, 1]` → `soma = 4 + (-1) + 2 + 1 = 6`

Repare: começar em -2 ou incluir o -3 e o -5 só diminuiria a soma total — por isso o melhor trecho "pula" esses números negativos isolados.

Implementação

```
def max_sub_array(nums):
    max_atual = nums[0]
    max_global = nums[0]

    for num in nums[1:]:
        if num > max_atual + num:
            max_atual = num
        else:
            max_atual += num

        if max_atual > max_global:
            max_global = max_atual

    return max_global
```

Como funciona

- Inicializa max_atual e max_global com o primeiro elemento do vetor.
- A cada elemento seguinte, decide: se o elemento sozinho supera a soma acumulada + elemento, recomeça; senão, estende.
- Compara max_atual com max_global para guardar o maior valor já visto.
- Ao final, max_global contém a resposta — nenhum laço extra, nenhuma estrutura auxiliar.

O ganho de usar Kadane

Abordagem	Tempo	Espaço	Limitação
Força bruta	$O(n^3)$	$O(1)$	Recalcula somas repetidamente
Soma de prefixos	$O(n^2)$	$O(n)$	Ainda testa todos os pares (i, j)
Kadane (ótimo)	**$O(n)$**	**$O(1)$**	**Uma única passada, sem memória extra**

Principal trade-off: abrimos mão de considerar todas as combinações possíveis para obter uma decisão ótima tomada em tempo constante por elemento.

ONDE O CONCEITO É UTILIZADO?

- Mercado financeiro: melhor janela de ganho em séries de preços
- Processamento de imagem: região de maior intensidade de pixels
- Séries temporais / sensores : intervalo de maior atividade em métricas
- Algoritmos de streaming: análise incremental sem reprocessar tudo

O algoritmo de Kadane mostra como reconhecer uma decisão local ótima transforma um problema combinatório em uma única passada linear.

Obrigado!